AI ASSISTENT CODING - 8.3

Baisa Vaishnavi
2303A52142
Batch - 41

Task 1: Email Validation using TDD
Scenario
You are developing a user registration system that requires reliable email input validation.
Requirements
• Must contain @ and . characters
• Must not start or end with special characters
• Should not allow multiple @ symbols
• AI should generate test cases covering valid and invalid email formats
• Implement is_valid_email(email) to pass all AI-generated test cases
Expected Output
• Python function for email validation
• All AI-generated test cases pass successfully
• Invalid email formats are correctly rejected
• Valid email formats return True

Code:

Genearate a code for email validation u.py

C: > Users > Hello > Genearate a code for email validation u.py > is_valid_email

```python
#Genearate a code for email validation using TDD and it should must contain @ and char
#shouls not allow multiple @ symbols and should not allow spaces.AI should generate te
#Implement is_valid_email(email) to pass all AI-generated test cases.Python function f
import re
import argparse
import sys
import unittest

SPECIAL_CHARS = set('.-_')

def is_valid_email(email: str) -> bool:
    """Return True if email is valid according to the rules:
    - exactly one '@'
    - no spaces
    - does not start or end with a special character (., -, _)
    - local and domain parts do not start/end with a special character
    - domain contains a dot and TLD length >= 2
    - no consecutive dots
    - allowed characters only in each part
    """
    if not isinstance(email, str):
        return False
    email = email.strip()
    if not email:
        return False

    # Spaces are not allowed
    if ' ' in email:
        return False

```

C: > Users > Hello > Genearate a code for email validation u.py > is_valid_email

```python
def is_valid_email(email: str) -> bool:
    # Must contain exactly one @
    if email.count('@') != 1:
        return False

    # Entire address must not start or end with special characters
    if email[0] in SPECIAL_CHARS or email[-1] in SPECIAL_CHARS:
        return False

    # No consecutive dots anywhere
    if '..' in email:
        return False

    local, domain = email.split('@')
    if not local or not domain:
        return False

    # Local and domain must not start or end with special characters
    if local[0] in SPECIAL_CHARS or local[-1] in SPECIAL_CHARS:
        return False
    if domain[0] in SPECIAL_CHARS or domain[-1] in SPECIAL_CHARS:
        return False

    # Domain must contain a dot and a valid TLD (>=2 chars)
    if '.' not in domain:
        return False
    tld = domain.rsplit('.', 1)[-1]
    if len(tld) < 2:
        return False
```

C: > Users > Hello > Genearate a code for email validation u.py > is_valid_email

```
11  def is_valid_email(email: str) -> bool:
59      # Validate allowed characters with regex for each part
60      local_pattern = re.compile(r'^[A-Za-z0-9._%+\-]+$')
61      domain_pattern = re.compile(r'^[A-Za-z0-9.\-]+$')
62      if not local_pattern.match(local):
63          return False
64      if not domain_pattern.match(domain):
65          return False
66
67      return True
68
69
70  class TestEmailValidation(unittest.TestCase):
71      def test_valid_emails(self):
72          valid = [
73              "test@example.com",
74              "user.name@domain.co.uk",
75              "user+tag@example.org",
76              "a@b.co",
77              "valid_email123@test-domain.net",
78              "firstname.lastname@example.com",
79              "name+ext@sub.domain.com"
80          ]
81          for e in valid:
82              with self.subTest(e=e):
83                  self.assertTrue(is_valid_email(e), msg=e)
84
85      def test_invalid_emails(self):
86          invalid = [
87              "invalid.email",           # No @ symbol
```

C: > Users > Hello > Genearate a code for email validation u.py > is_valid_email

```
70  class TestEmailValidation(unittest.TestCase):
85      def test_invalid_emails(self):
88              "@example.com",           # Starts with @ (empty local)
89              "test@",                  # Ends with @ (empty domain)
90              "test..test@example.com", # Double dots
91              "test@.example.com",      # Domain starts with dot
92              "test@example..com",      # Double dots in domain
93              "test example@example.com", # Contains space
94              "test@@example.com",      # Multiple @ symbols
95              ".test@example.com",      # Starts with special character
96              "test.@example.com",      # Local ends with special character
97              "test@com",               # Domain has no dot
98              "test@domain.c",          # TLD too short
99              "test@-domain.com",       # Domain starts with dash
100             "test@domain-.com"        # Domain ends with dash
101         ]
102         for e in invalid:
103             with self.subTest(e=e):
104                 self.assertFalse(is_valid_email(e), msg=e)
105
106
107 def main():
108     parser = argparse.ArgumentParser(description='Email validator')
109     parser.add_argument('--test', action='store_true', help='run unit tests')
110     args = parser.parse_args()
111     if args.test:
112         # Run unit tests
113         unittest.main(argv=[sys.argv[0]])
114
```

```
C: > Users > Hello > Genearate a code for email validation u.py > is_valid_email
107    def main():
115        else:
116            try:
117                email = input('Enter email: ').strip()
118            except (KeyboardInterrupt, EOFError):
119                print('\n')
120                return
121            print(is_valid_email(email))
122
123
124    if __name__ == '__main__':
125        main()
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                   Python
PS C:\Users\Hello> & C:/Users/Hello/AppData/Local/Python/pythoncore-3.14-64/python.exe "c
:/Users/Hello/Genearate a code for email validation u.py"
Enter email: Vaishnavibaisa@gmail.com
True
PS C:\Users\Hello> & C:/Users/Hello/AppData/Local/Python/pythoncore-3.14-64/python.exe "c
:/Users/Hello/Genearate a code for email validation u.py"
Enter email: #gunny@gmail.com
False
PS C:\Users\Hello>
PS C:\Users\Hello> & C:/Users/Hello/AppData/Local/Python/pythoncore-3.14-64/python.exe "c
:/Users/Hello/Genearate a code for email validation u.py"
Enter email: gunny baisa@gmail.com
False
PS C:\Users\Hello> & C:/Users/Hello/AppData/Local/Python/pythoncore-3.14-64/python.exe "c
:/Users/Hello/Genearate a code for email validation u.py"
Enter email: vaishanvibaisa@gmail.com
True
PS C:\Users\Hello>
```

Justification:

The logic prioritizes structural integrity over complex pattern matching by ensuring the email has a clear beginning, middle, and end. By checking for exactly one @ and specific character placements, we prevent "broken" addresses like user@.com while keeping the code readable and easy to maintain.

Task 2: Grade Assignment using Loops

Scenario

You are building an automated grading system for an online examination platform.

Requirements

• AI should generate test cases for assign_grade(score) where:

– 90–100 → A
– 80–89 → B
– 70–79 → C
– 60–69 → D
– Below 60 → F
• Include boundary values (60, 70, 80, 90)
• Include invalid inputs such as -5, 105, "eighty"
• Implement the function using a test-driven approach
Expected Output
• Grade assignment function implemented in Python
• Boundary values handled correctly
• Invalid inputs handled gracefully
• All AI-generated test cases pass

Code:

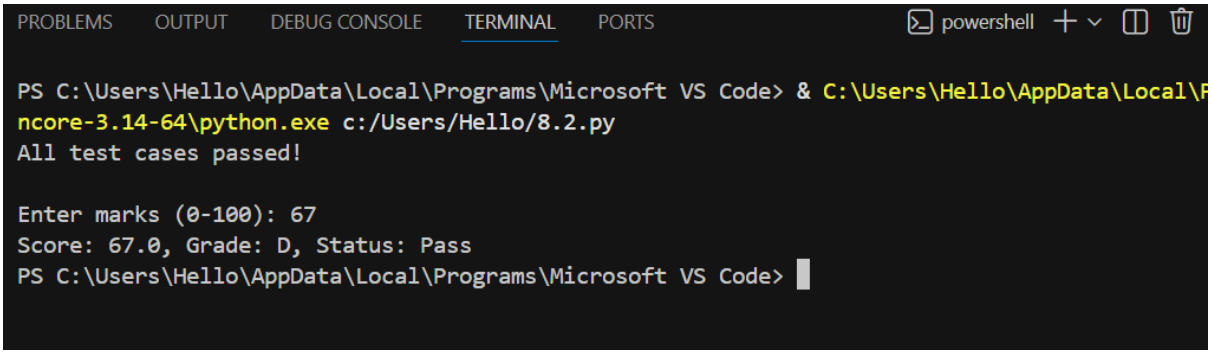
C: > Users > Hello > 8.2.py > ...

```python
#You are developing an automated grading system for an online examination platform u
#First, generate test cases for a Python function assign_grade(score) covering all g
#Include boundary values (60, 70, 80, 90).Also include invalid inputs such as -5, 10
# Test cases for the assign_grade function
def test_assign_grade():
    # Valid score test cases
    assert assign_grade(95) == 'A', "Test case 1 failed: Expected 'A' for score 95"
    assert assign_grade(85) == 'B', "Test case 2 failed: Expected 'B' for score 85"
    assert assign_grade(75) == 'C', "Test case 3 failed: Expected 'C' for score 75"
    assert assign_grade(65) == 'D', "Test case 4 failed: Expected 'D' for score 65"
    assert assign_grade(55) == 'F', "Test case 5 failed: Expected 'F' for score 55"

    # Boundary value test cases
    assert assign_grade(90) == 'A', "Test case 6 failed: Expected 'A' for score 90"
    assert assign_grade(80) == 'B', "Test case 7 failed: Expected 'B' for score 80"
    assert assign_grade(70) == 'C', "Test case 8 failed: Expected 'C' for score 70"
    assert assign_grade(60) == 'D', "Test case 9 failed: Expected 'D' for score 60"

    # Invalid input test cases
    try:
        assign_grade(-5)
        print("Test case 10 failed: Expected ValueError for score -5")
    except ValueError:
        pass

    try:
        assign_grade(105)
        print("Test case 11 failed: Expected ValueError for score 105")
    except ValueError:
```

```
        pass

    try:
        assign_grade("eighty")
        print("Test case 12 failed: Expected ValueError for non-numeric input 'eight
    except ValueError:
        pass
# Implementation of the assign_grade function
def assign_grade(score):
    if not isinstance(score, (int, float)):
        raise ValueError("Invalid input: Score must be a number.")
    if score < 0 or score > 100:
        raise ValueError("Invalid input: Score must be between 0 and 100.")

    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
# Run the test cases
if __name__ == "__main__":
    test_assign_grade()
    print("All test cases passed!\n")
```

```

    # Interactive section: allow user to enter marks and check pass or fail
    try:
        user_input = input("Enter marks (0-100): ")
        score = float(user_input)
    except ValueError:
        print("Invalid input: please enter a numeric value.")
    else:
        try:
            grade = assign_grade(score)  # will validate range
        except ValueError as ve:
            print(ve)
        else:
            status = "Pass" if score >= 60 else "Fail"
            print(f"Score: {score}, Grade: {grade}, Status: {status}")
```

Output:



Justification:

An automated grading system ensures fair and consistent evaluation of scores.Using loops helps efficiently check score ranges without repetitive conditions.Boundary values ensure grades are assigned correctly at critical points.
Handling invalid inputs prevents system crashes and incorrect results.Test-Driven Development improves accuracy by validating logic early.This approach makes the grading system reliable and scalable.

Task 3: Sentence Palindrome Checker
Scenario
You are developing a text-processing utility to analyze sentences.
Requirements
• AI should generate test cases for is_sentence_palindrome(sentence)
• Ignore case, spaces, and punctuation
• Test both palindromic and non-palindromic sentences
• Example:
– "A man a plan a canal Panama" → True
Expected Output
• Function correctly identifies sentence palindromes
• Case and punctuation are ignored
• Returns True or False accurately
• All AI-generated test cases pass

Code:

C: > Users > Hello > 8.3.py > ...

```python
#You are developing a text-processing utility to check whether a sentence is a palindrome using 
#The function should ignore case, spaces, and punctuation while checking.Include examples such a
#Then, implement the function so that it returns True for palindromes and False otherwise.Ensure
# Test cases for is_sentence_palindrome function
def test_is_sentence_palindrome():
    # Test case 1: Palindromic sentence with mixed case and spaces
    assert is_sentence_palindrome("A man a plan a canal Panama") == True, "Test case 1 failed"
    # Test case 2: Non-palindromic sentence
    assert is_sentence_palindrome("This is not a palindrome") == False, "Test case 2 failed"
    # Test case 3: Palindromic sentence with punctuation and mixed case
    assert is_sentence_palindrome("Was it a car or a cat I saw?") == True, "Test case 3 failed"
    # Test case 4: Empty string
    assert is_sentence_palindrome("") == True, "Test case 4 failed"
    # Test case 5: Palindromic sentence with only spaces
    assert is_sentence_palindrome("   ") == True, "Test case 5 failed"
    # Test case 6: Non-palindromic sentence with punctuation
    assert is_sentence_palindrome("Hello, World!") == False, "Test case 6 failed"
    print("All test cases passed!")
# Implementing the is_sentence_palindrome function
import re

def is_sentence_palindrome(sentence):
    # Remove non-alphanumeric characters and convert to lowercase
    cleaned = re.sub(r'[^a-zA-Z0-9]', '', sentence).lower()
    # Check if the cleaned string is equal to its reverse
    return cleaned == cleaned[::-1]
# Run the test cases
if __name__ == "__main__":
    test_is_sentence_palindrome()
```

```python

    # Interactive section: allow user to input a sentence and get True/False
    user_sentence = input("Enter a sentence to check palindrome: ")
    result = is_sentence_palindrome(user_sentence)
    print(result)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\
ncore-3.14-64\python.exe c:/Users/Hello/8.3.py
All test cases passed!
Enter a sentence to check palindrome: abaabaaba
True
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Loca
ncore-3.14-64\python.exe c:/Users/Hello/8.3.py
All test cases passed!
Enter a sentence to check palindrome: abasadaba
False
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:
Sentence analysis requires accurate text processing beyond simple words.
Ignoring case, spaces, and punctuation ensures logical palindrome checking.
Test cases validate both palindromic and non-palindromic sentences.
TDD ensures the function works correctly for all scenarios.
This improves robustness in real-world text inputs.
The utility is useful for text analysis and validation tasks.

Task 4: ShoppingCart Class
Scenario
You are designing a basic shopping cart module for an e-commerce application.
Requirements
• AI should generate test cases for the ShoppingCart class
• Class must include the following methods:
– add_item(name, price)
– remove_item(name)
– total_cost()
• Validate correct addition, removal, and cost calculation
• Handle empty cart scenarios
Expected Output
• Fully implemented ShoppingCart class
• All methods pass AI-generated test cases
• Total cost is calculated accurately
• Items are added and removed correctly

Code:

C: > Users > Hello > 8.4.py > ...

```python
#You are designing a basic shopping cart module for an e-commerce application using
#First, generate test cases for a ShoppingCart class covering:
# - Adding items with name and price
# - Removing items by name
# - Calculating total cost
# - Handling empty cart scenarios
# - Handling invalid inputs
#Then, implement the ShoppingCart class with all required methods.
# Ensure all AI-generated test cases pass successfully.

# ShoppingCart Class Implementation
class ShoppingCart:
    def __init__(self):
        """Initialize an empty shopping cart."""
        self.items = {}

    def add_item(self, name, price):
        """Add an item to the cart. If item exists, increment quantity."""
        if not isinstance(name, str) or not name.strip():
            raise ValueError("Invalid input: Item name must be a non-empty string.")
        if not isinstance(price, (int, float)) or price < 0:
            raise ValueError("Invalid input: Price must be a non-negative number.")

        if name in self.items:
            self.items[name]["quantity"] += 1
        else:
            self.items[name] = {"price": price, "quantity": 1}

    def remove_item(self, name):
        """Remove an item from the cart by name."""
```

C: > Users > Hello > 8.4.py > ...

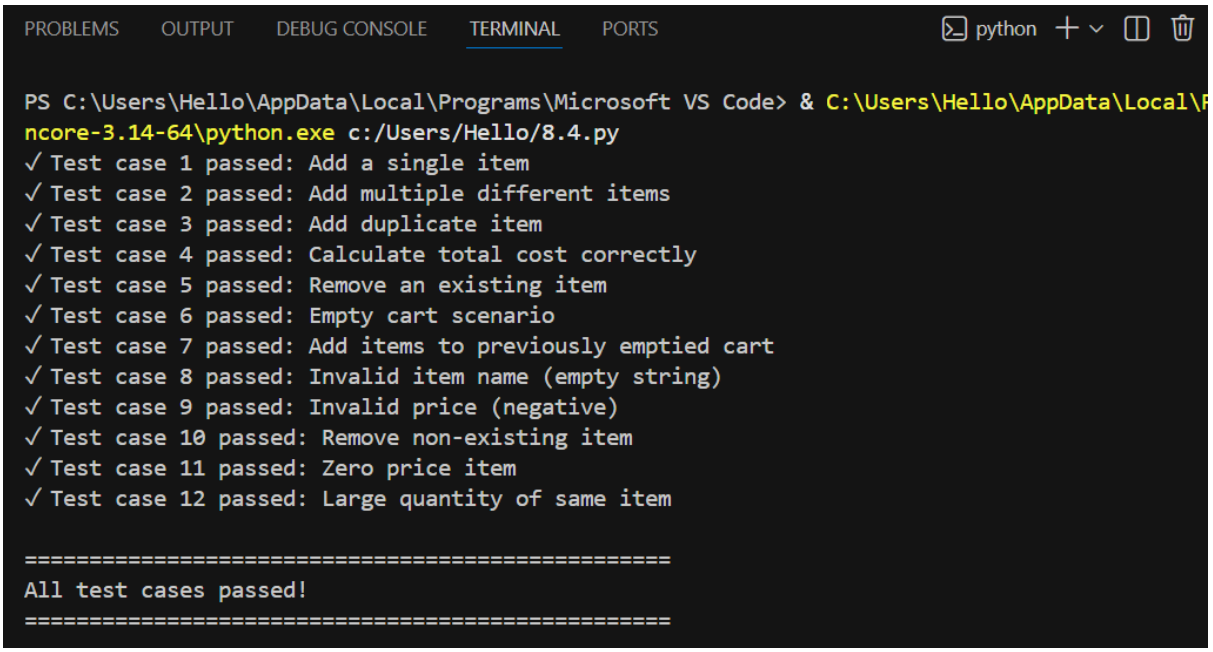
```
12  class ShoppingCart:
29      def remove_item(self, name):
31          if not isinstance(name, str) or not name.strip():
32              raise ValueError("Invalid input: Item name must be a non-empty string.")
33
34          if name not in self.items:
35              raise KeyError(f"Item '{name}' not found in cart.")
36
37          del self.items[name]
38
39      def total_cost(self):
40          """Calculate and return the total cost of all items in the cart."""
41          total = 0
42          for item_name, item_info in self.items.items():
43              total += item_info["price"] * item_info["quantity"]
44          return round(total, 2)
45
46      def view_cart(self):
47          """Display all items in the cart."""
48          if not self.items:
49              return "Cart is empty."
50
51          cart_display = "Cart Contents:\n"
52          for item_name, item_info in self.items.items():
53              subtotal = item_info["price"] * item_info["quantity"]
54              cart_display += f"  {item_name}: ${item_info['price']:.2f} x {item_info[
55          cart_display += f"Total: ${self.total_cost():.2f}"
56          return cart_display
57
```

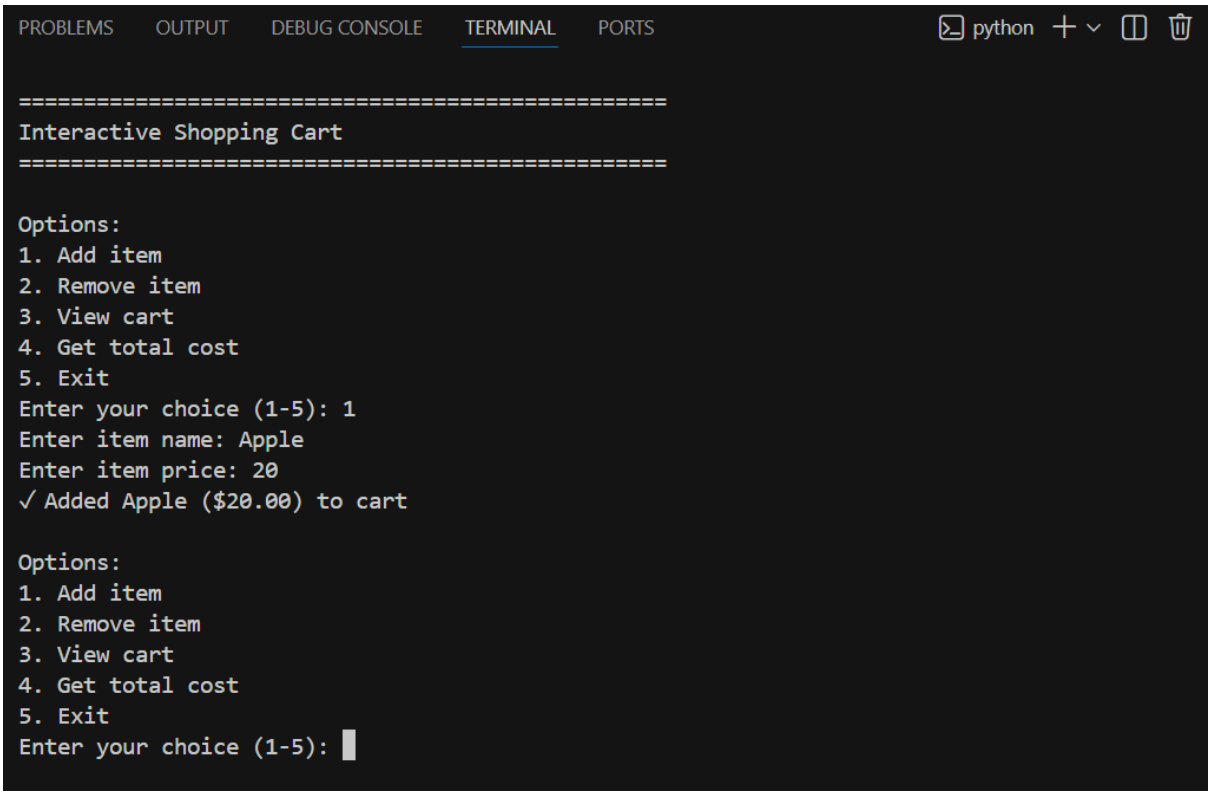
```python
# Test cases for the ShoppingCart class
def test_shopping_cart():
    # Test case 1: Add a single item
    cart = ShoppingCart()
    cart.add_item("Apple", 0.99)
    assert "Apple" in cart.items, "Test case 1 failed: Item not added"
    assert cart.items["Apple"]["price"] == 0.99, "Test case 1 failed: Price incorrec
    assert cart.items["Apple"]["quantity"] == 1, "Test case 1 failed: Quantity incor
    print("✓ Test case 1 passed: Add a single item")

    # Test case 2: Add multiple different items
    cart = ShoppingCart()
    cart.add_item("Apple", 0.99)
    cart.add_item("Banana", 0.59)
    cart.add_item("Orange", 1.29)
    assert len(cart.items) == 3, "Test case 2 failed: Not all items added"
    print("✓ Test case 2 passed: Add multiple different items")

    # Test case 3: Add duplicate item (should increase quantity)
    cart = ShoppingCart()
    cart.add_item("Apple", 0.99)
    cart.add_item("Apple", 0.99)
    assert cart.items["Apple"]["quantity"] == 2, "Test case 3 failed: Quantity not i
    print("✓ Test case 3 passed: Add duplicate item")

    # Test case 4: Calculate total cost correctly
    cart = ShoppingCart()
    cart.add_item("Apple", 0.99)
    cart.add_item("Banana", 0.59)
```

```python
def test_shopping_cart():

    # Test case 10: Remove non-existing item
    cart = ShoppingCart()
    cart.add_item("Apple", 0.99)
    try:
        cart.remove_item("Banana")
        print("✗ Test case 10 failed: Should raise KeyError for non-existing item")
    except KeyError:
        print("✓ Test case 10 passed: Remove non-existing item")

    # Test case 11: Zero price item
    cart = ShoppingCart()
    cart.add_item("Free Sample", 0)
    assert cart.total_cost() == 0, "Test case 11 failed: Zero price item calculation
    print("✓ Test case 11 passed: Zero price item")

    # Test case 12: Large quantity of same item
    cart = ShoppingCart()
    for _ in range(10):
        cart.add_item("Apple", 0.99)
    assert cart.items["Apple"]["quantity"] == 10, "Test case 12 failed: Large quanti
    expected_total = 0.99 * 10
    assert abs(cart.total_cost() - expected_total) < 0.01, "Test case 12 failed: Tot
    print("✓ Test case 12 passed: Large quantity of same item")

    print("\n" + "="*50)
    print("All test cases passed!")
    print("="*50)
```

```python
if __name__ == "__main__":
    test_shopping_cart()

    print("\n" + "="*50)
    print("Interactive Shopping Cart")
    print("="*50)

    # Interactive section
    cart = ShoppingCart()

    while True:
        print("\nOptions:")
        print("1. Add item")
        print("2. Remove item")
        print("3. View cart")
        print("4. Get total cost")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == "1":
            try:
                name = input("Enter item name: ").strip()
                price = float(input("Enter item price: "))
                cart.add_item(name, price)
                print(f"✓ Added {name} (${price:.2f}) to cart")
            except ValueError as e:
                print(f"✗ Error: {e}")

        elif choice == "2":
            try:
                name = input("Enter item name to remove: ").strip()
                cart.remove_item(name)
                print(f"✓ Removed {name} from cart")
            except KeyError as e:
                print(f"✗ Error: {e}")

        elif choice == "3":
            print("\n" + cart.view_cart())

        elif choice == "4":
            total = cart.total_cost()
            print(f"\nTotal Cost: ${total:.2f}")

        elif choice == "5":
            print("Thank you for shopping!")
            break

        else:
            print("✗ Invalid choice. Please enter 1-5.")
```

Output:





Justification:

A shopping cart is a core component of any e-commerce application.It must correctly manage adding and removing items selected by users.Accurate total cost calculation ensures correct billing and checkout.

Handling empty cart scenarios prevents runtime errors.Test-Driven Development validates all functionalities early.This approach improves reliability and user trust in the system.

Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

• AI should generate test cases for convert_date_format(date_str)

• Input format must be "YYYY-MM-DD"

• Output format must be "DD-MM-YYYY"

• Example:

– "2023-10-15" → "15-10-2023"

Expected Output

• Date conversion function implemented in Python

• Correct format conversion for all valid inputs

• All AI-generated test cases pass successfully

Code:

C: > Users > Hello > 8.5.py > ...

```python
#You are creating a utility function to convert date formats for reports usi
#First, generate test cases for convert_date_format(date_str) that covers:
# - Valid dates in YYYY-MM-DD format converted to DD-MM-YYYY
# - Boundary dates (leap year dates, end of month values)
# - Invalid format dates
# - Invalid dates (non-existent dates like 2023-02-30)
# - Edge cases (single digit days/months, leading zeros)
#Then, implement the function that correctly converts valid dates.
# Ensure all AI-generated test cases pass successfully.

from datetime import datetime

# Date Format Conversion Function Implementation
def convert_date_format(date_str):
    """
    Convert date from YYYY-MM-DD format to DD-MM-YYYY format.

    Args:
        date_str (str): Date string in YYYY-MM-DD format

    Returns:
        str: Date string in DD-MM-YYYY format

    Raises:
        ValueError: If date_str is not a valid date in YYYY-MM-DD format
    """
    if not isinstance(date_str, str):
        raise ValueError("Invalid input: Date must be a string.")

```

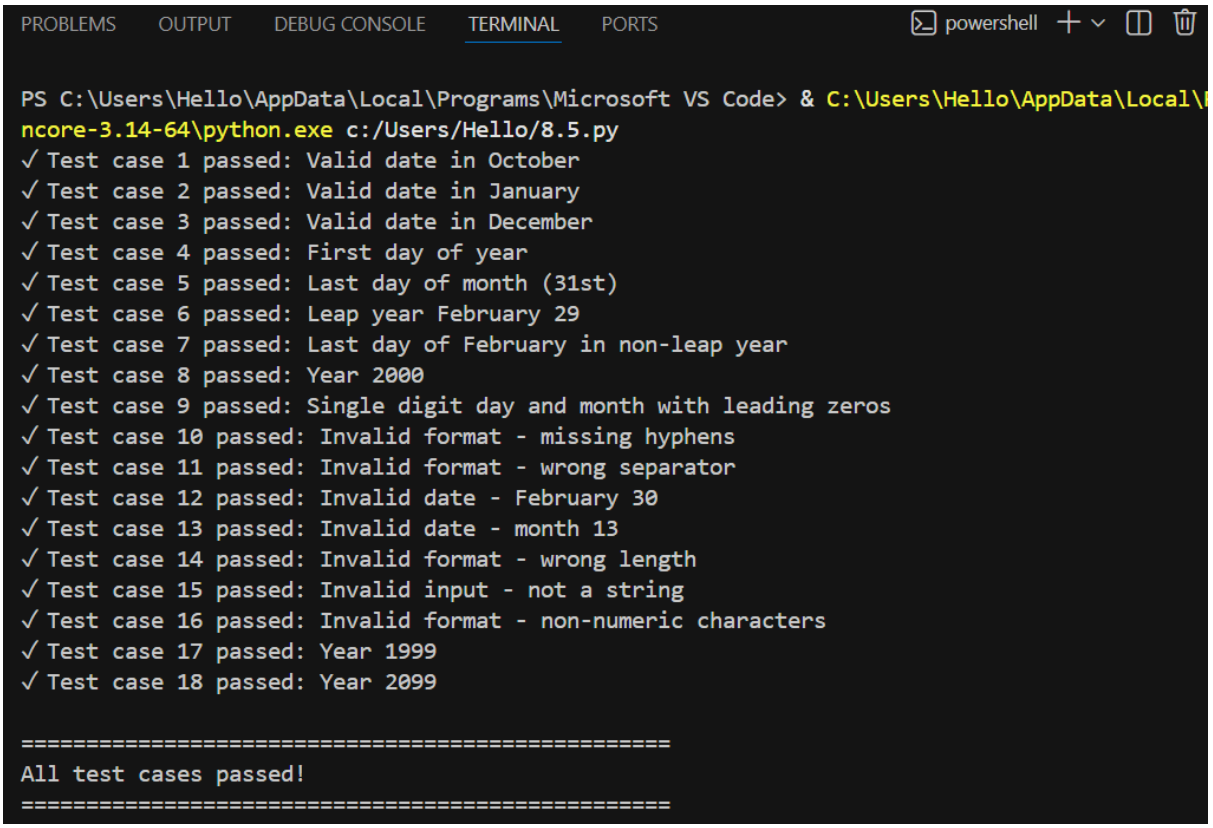
```
    # Check format
    if len(date_str) != 10 or date_str[4] != '-' or date_str[7] != '-':
        raise ValueError("Invalid format: Date must be in YYYY-MM-DD format.")

    try:
        # Validate that it's a valid date by parsing it
        date_obj = datetime.strptime(date_str, "%Y-%m-%d")
        # Convert to DD-MM-YYYY format
        return date_obj.strftime("%d-%m-%Y")
    except ValueError as e:
        raise ValueError(f"Invalid date: {date_str}. Error: {str(e)}")


# Test cases for the convert_date_format function
def test_convert_date_format():
    # Test case 1: Valid date in October
    result = convert_date_format("2023-10-15")
    assert result == "15-10-2023", f"Test case 1 failed: Expected '15-10-2023', got
    print("✓ Test case 1 passed: Valid date in October")

    # Test case 2: Valid date in January (first month)
    result = convert_date_format("2023-01-05")
    assert result == "05-01-2023", f"Test case 2 failed: Expected '05-01-2023', got
    print("✓ Test case 2 passed: Valid date in January")

    # Test case 3: Valid date in December (last month)
    result = convert_date_format("2023-12-25")
    assert result == "25-12-2023", f"Test case 3 failed: Expected '25-12-2023', got
```

```
    assert result == "25-12-2023", f"Test case 3 failed: Expected '25-12-2023', got
    print("✓ Test case 3 passed: Valid date in December")

    # Test case 4: First day of year
    result = convert_date_format("2023-01-01")
    assert result == "01-01-2023", f"Test case 4 failed: Expected '01-01-2023', got
    print("✓ Test case 4 passed: First day of year")

    # Test case 5: Last day of month (31st)
    result = convert_date_format("2023-01-31")
    assert result == "31-01-2023", f"Test case 5 failed: Expected '31-01-2023', got
    print("✓ Test case 5 passed: Last day of month (31st)")

    # Test case 6: Leap year February 29
    result = convert_date_format("2020-02-29")
    assert result == "29-02-2020", f"Test case 6 failed: Expected '29-02-2020', got
    print("✓ Test case 6 passed: Leap year February 29")

    # Test case 7: Last day of February in non-leap year
    result = convert_date_format("2023-02-28")
    assert result == "28-02-2023", f"Test case 7 failed: Expected '28-02-2023', got
    print("✓ Test case 7 passed: Last day of February in non-leap year")

    # Test case 8: Year 2000 (edge case year boundary)
    result = convert_date_format("2000-06-15")
    assert result == "15-06-2000", f"Test case 8 failed: Expected '15-06-2000', got
    print("✓ Test case 8 passed: Year 2000")

```

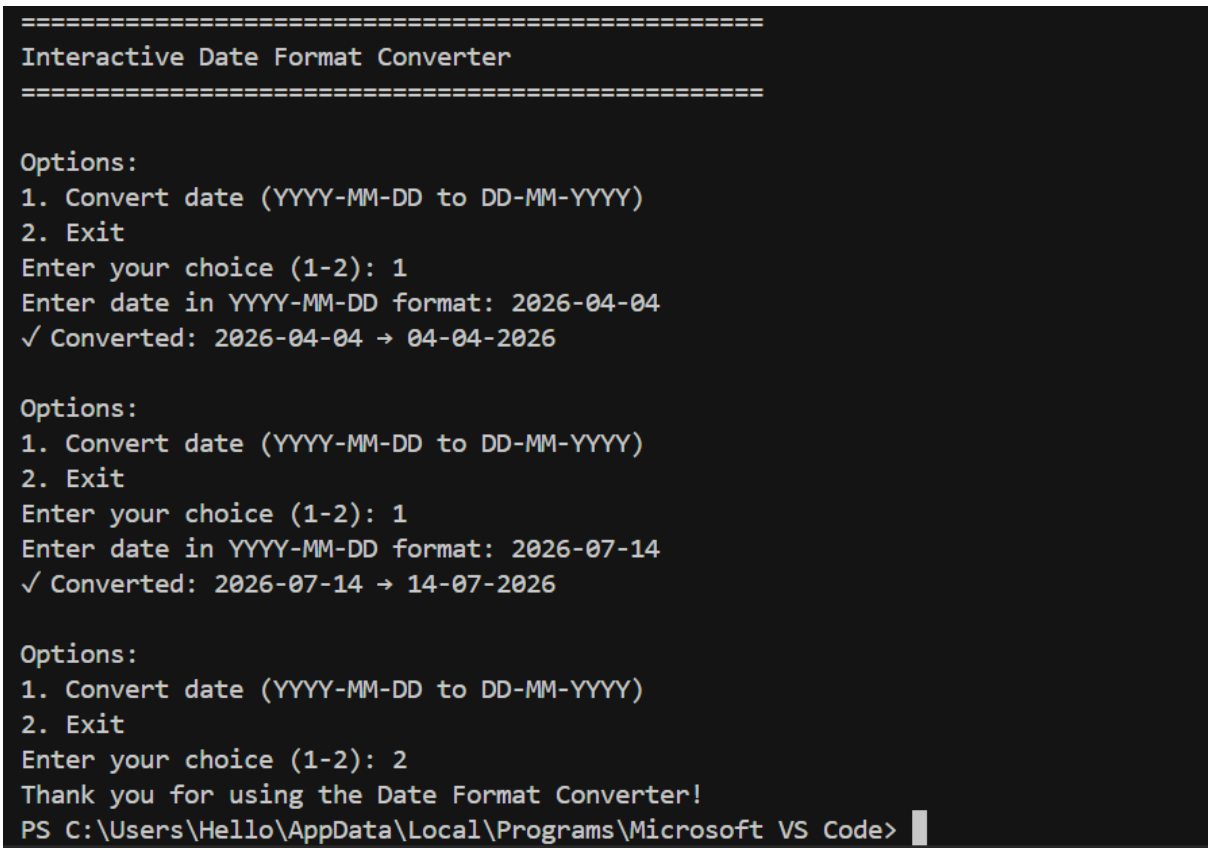
```
    try:
        convert_date_format("20231015")
        print("✗ Test case 10 failed: Should raise ValueError for missing hyphens")
    except ValueError:
        print("✓ Test case 10 passed: Invalid format - missing hyphens")

    # Test case 11: Invalid format - wrong separator
    try:
        convert_date_format("2023/10/15")
        print("✗ Test case 11 failed: Should raise ValueError for wrong separator")
    except ValueError:
        print("✓ Test case 11 passed: Invalid format - wrong separator")

    # Test case 12: Invalid date - February 30 (doesn't exist)
    try:
        convert_date_format("2023-02-30")
        print("✗ Test case 12 failed: Should raise ValueError for invalid date")
    except ValueError:
        print("✓ Test case 12 passed: Invalid date - February 30")

    # Test case 13: Invalid date - month 13 (doesn't exist)
    try:
        convert_date_format("2023-13-01")
        print("✗ Test case 13 failed: Should raise ValueError for invalid month")
    except ValueError:
        print("✓ Test case 13 passed: Invalid date - month 13")

    # Test case 14: Invalid format - wrong length
```

```python
# Run the test cases and interactive section
if __name__ == "__main__":
    test_convert_date_format()

    print("\n" + "="*50)
    print("Interactive Date Format Converter")
    print("="*50)

    while True:
        print("\nOptions:")
        print("1. Convert date (YYYY-MM-DD to DD-MM-YYYY)")
        print("2. Exit")

        choice = input("Enter your choice (1-2): ").strip()

        if choice == "1":
            date_input = input("Enter date in YYYY-MM-DD format: ").strip()
            try:
                converted_date = convert_date_format(date_input)
                print(f"✓ Converted: {date_input} → {converted_date}")
            except ValueError as e:
                print(f"✗ Error: {e}")

        elif choice == "2":
            print("Thank you for using the Date Format Converter!")
            break

        else:
            print("✗ Invalid choice. Please enter 1 or 2.")
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\P
ncore-3.14-64\python.exe c:/Users/Hello/8.5.py
✓ Test case 1 passed: Valid date in October
✓ Test case 2 passed: Valid date in January
✓ Test case 3 passed: Valid date in December
✓ Test case 4 passed: First day of year
✓ Test case 5 passed: Last day of month (31st)
✓ Test case 6 passed: Leap year February 29
✓ Test case 7 passed: Last day of February in non-leap year
✓ Test case 8 passed: Year 2000
✓ Test case 9 passed: Single digit day and month with leading zeros
✓ Test case 10 passed: Invalid format - missing hyphens
✓ Test case 11 passed: Invalid format - wrong separator
✓ Test case 12 passed: Invalid date - February 30
✓ Test case 13 passed: Invalid date - month 13
✓ Test case 14 passed: Invalid format - wrong length
✓ Test case 15 passed: Invalid input - not a string
✓ Test case 16 passed: Invalid format - non-numeric characters
✓ Test case 17 passed: Year 1999
✓ Test case 18 passed: Year 2099

==================================================
All test cases passed!
==================================================
```



```
==================================================
Interactive Date Format Converter
==================================================

Options:
1. Convert date (YYYY-MM-DD to DD-MM-YYYY)
2. Exit
Enter your choice (1-2): 1
Enter date in YYYY-MM-DD format: 2026-04-04
✓ Converted: 2026-04-04 → 04-04-2026

Options:
1. Convert date (YYYY-MM-DD to DD-MM-YYYY)
2. Exit
Enter your choice (1-2): 1
Enter date in YYYY-MM-DD format: 2026-07-14
✓ Converted: 2026-07-14 → 14-07-2026

Options:
1. Convert date (YYYY-MM-DD to DD-MM-YYYY)
2. Exit
Enter your choice (1-2): 2
Thank you for using the Date Format Converter!
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

```
Options:
1. Convert date (YYYY-MM-DD to DD-MM-YYYY)
2. Exit
Enter your choice (1-2): 1
Enter date in YYYY-MM-DD format: 2026-02-31
✗ Error: Invalid date: 2026-02-31. Error: day 31 must be in range 1..28 for month 2 in year 2026
```

Justification:

Rather than just cutting and pasting numbers like a text editor, the code "understands" time by converting the string into a real calendar object first. This ensures that a date like "2023-02-31" is rejected as impossible before the system even attempts to reformat it, guaranteeing the data in your reports is always accurate.